

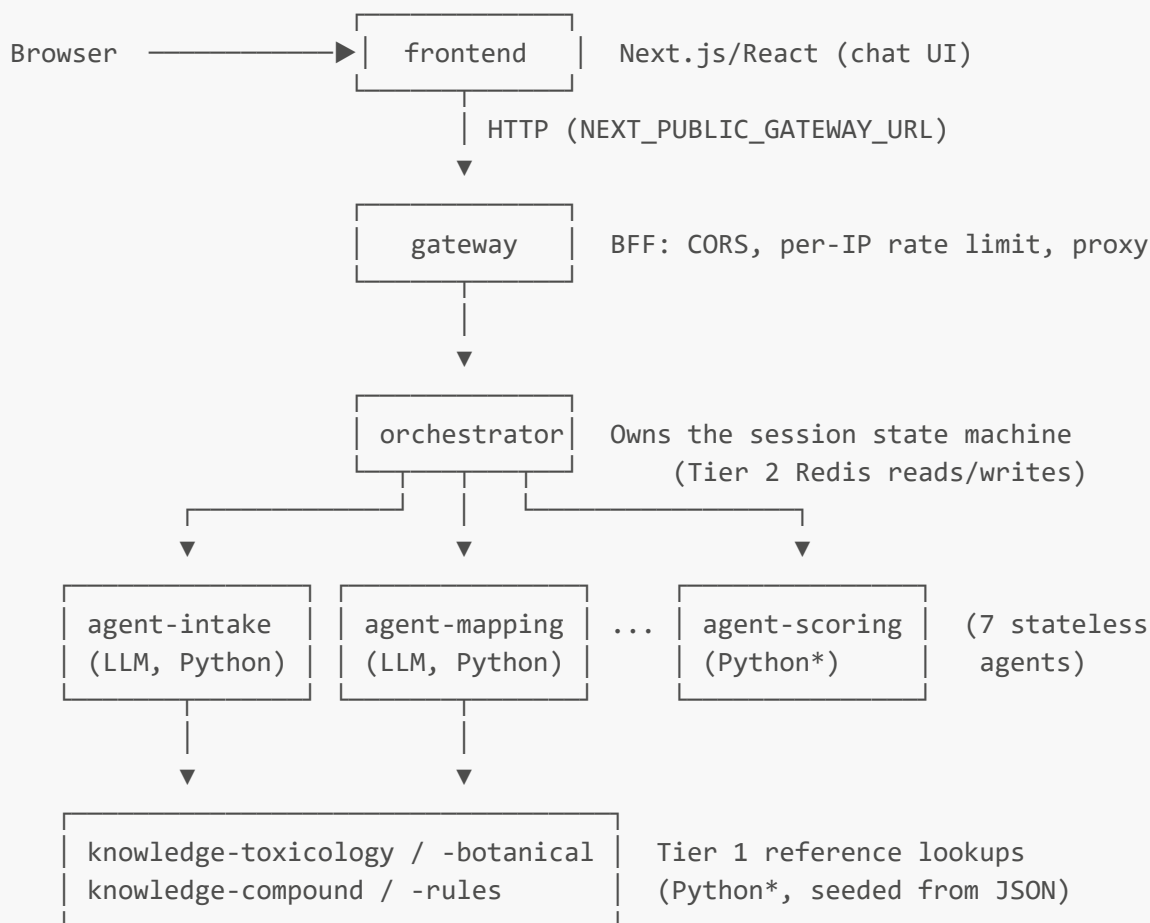
Rootwell — Architecture

This is the as-built system architecture: what actually runs today, how a request flows through it, and the structural decisions that aren't obvious from reading any single file. For the original product-vision documents (scoring formula derivation, UX flow, full design rationale) see [research docs/application_design.md](#) (v1) and [research docs/application_design_v2_microservices_agentic.md](#) (v2). This document is the "current state + why" companion to those.

1. One-paragraph summary

Rootwell is a conversational web app that takes user-reported symptoms, traces them to plausible biochemical root causes, and recommends herbs with some evidence behind them. It is built as ~13 independently deployable backend services plus a Next.js frontend, all coordinated by one orchestrator, with **Redis as the only stateful component in the entire system** — there is no SQL/NoSQL database anywhere. The backend is mid-migration from all-Python to a polyglot split: services that call an LLM stay in Python (FastAPI); every other service is being rewritten in Java (Spring Boot). See §7 for migration status.

2. Service map



plus: email service (Resend/mock), called by orchestrator at the end-of-session export step.

* = migrating to Java; see §7. Redis backs every Tier 1/2 read+write.

Every backend service exposes `GET /healthz`. The gateway is the *only* thing the frontend ever calls directly — no service is reachable from the browser except through it.

Why this decomposition

Each agent maps to one accountable decision in the recommendation pipeline (intake → mapping → retrieval → safety → scoring → explanation → reporting), and each boundary was drawn where the **trust model changes**:

- Safety verdicts must never depend on an LLM's judgment (a hallucinated "this is fine" is a real-harm scenario), so `agent-safety` is deterministic and structurally cannot call an LLM — it doesn't even import the LLM wrapper.
- Scoring is a fixed weighted formula (see `research docs/application_design_v2...md §7`), not model output, so it's independently testable and auditable.
- Only three services touch an LLM at all (`agent-intake`, `agent-mapping`, `agent-explanation`) — everywhere else, "why was this recommended" traces back to code and data, not a model call.

3. Data model: no database, Redis only

There is no SQL/NoSQL database in this system by design. Redis is split into two tiers with very different lifecycles:

Tier	Key pattern	Contents	Lifecycle
Tier 1 (reference)	<code>ref:*</code>	Herb/compound/symptom/rule records, no PII	Loaded from <code>seed/data/*.json</code> at service startup, 6h TTL, shared across every session
Tier 2 (session)	<code>session: {sid}:*</code>	One user's chat, symptom/cause cache, recommendations	TTL'd (idle timeout, default 30 min), hard-deleted (UNLINK) the moment a session ends or the user emails themselves the results

All access goes through `shared/shared/cache.py` (Python side) — nothing talks to Redis directly except that one module and, on the Java side, the equivalent `RefCacheService` per migrated service (read-only for Tier 1; Tier 2 read/write stays exclusively in the orchestrator, which stays Python regardless of migration since it's the one place session state is mutated).

Why no database at all, not even for Tier 1 reference data: the Tier 1 dataset is small (~18 herbs, 20 symptoms, 18 compounds, 25 rules), fully rebuildable from JSON in milliseconds, and read far more often than written. A database would add an operational dependency (schema migrations, connection pooling, a second technology to run locally) for data that fits comfortably in memory and doesn't need relational querying — every access pattern here is "fetch by id" or "list all of kind X," which a Redis string/hash already serves in $O(1)/O(n)$ with no query planner needed. If the real curated dataset (Phase 2) grows into something needing

relational queries (e.g. "all herbs with contraindication X interacting with compound Y"), that's the point to introduce Postgres — not before.

4. Request flow: a full session, end to end

1. **POST /session** (gateway → orchestrator) — orchestrator mints a `uuid4().hex` session id, creates the Tier 2 hash/stream keys, asks `agent-intake` for a greeting, and returns `{session_id, greeting}`. The session id itself is the bearer credential — there's no login, no account system; possession of the id is the capability. This is a deliberate scope decision, not an oversight (see `research/docs/application_design_v2...md` for the reasoning) — there's nothing to authenticate against because nothing is tied to a real identity until the optional email step.
2. **Symptom/cause collection** — each `POST /session/{sid}/message` is proxied to the orchestrator, which routes to `agent-intake`'s `symptom-turn` or `cause-turn` endpoint depending on `current_step`. Intake either calls Claude (real mode) or runs a deterministic keyword-matching fallback (mock mode) — the response shape is identical either way, so nothing downstream needs to know which mode is active. Matched items are cached; *suggested* items are surfaced but require an explicit `/add-item` call — the model/heuristic never silently adds something the user didn't confirm.
3. **POST /session/{sid}/analyze** — the orchestrator runs the full pipeline in sequence: `agent-mapping` (symptoms → candidate biochemical imbalances) → `agent-retrieval` (imbalances/symptoms → candidate herbs, by calling `knowledge-botanical` + `knowledge-compound`) → `agent-safety` (candidates + volunteered profile → per-herb allow/safety-factor verdicts, via `knowledge-rules`) → `agent-scoring` (candidates + verdicts → ranked, safety-adjusted scores) → `agent-explanation` (ranked list → final natural-language recommendations, LLM or template). Each hop is a plain HTTP POST with a JSON body/response — no shared in-memory state, no message queue; the orchestrator is a synchronous fan-out/fan-in caller, not an event bus.
4. **POST /session/{sid}/email/request → /email/confirm** — a verification-code gate (10-minute TTL, 3 requests/hour per address) sits in front of actually sending anything, specifically so the "email me this" feature can't be used as an open relay to spam arbitrary addresses. On confirm, `agent-reporting` deterministically templates the full transcript + recommendations into subject/html/text (no LLM — the disclaimer text and structure must not vary), the `email` service sends it (or mock-logs it), and the orchestrator immediately calls `purge_session` — Tier 2 data for that session is gone the instant the export succeeds.
5. **POST /session/{sid}/end** — same hard purge, for a user who just walks away without exporting.

5. Cross-cutting design rules (structural, not just documented)

- **Never proactively ask for personal details.** There is no profile-collection step anywhere in the orchestrator's state graph (§4.2 above has no such state). Age/pregnancy/medications/allergies/conditions are only ever populated via `agent-intake`'s extraction pass over free text the user volunteered unprompted. This is enforced by the state machine's shape, not a prompt instruction that could be worked around.
- **Safety is independent of scoring, and both are independent of the LLM.** `agent-safety` and `agent-scoring` do not import the LLM wrapper at all — verified structurally, not just by convention (see `docs/TECHNICAL_GUIDE.md` for how this was confirmed for the Java migration's scope split too).
- **Mock mode is a first-class runtime mode, not a test stub.** Every service that calls an external API (`shared/shared/llm.py` for Anthropic, `services/email/main.py` for Resend) checks for the corresponding key at import time and switches to a clearly-labeled deterministic fallback if it's absent

— same response shape, same endpoints, zero code branching required by callers. This is what lets the whole system run end-to-end with zero external credentials.

- **Curation status is threaded through, not stripped.** Every herb record carries `curation_status: "starter_dataset_unreviewed"`, and that field survives all the way to the final recommendation and the emailed report — a starter/demo dataset is never allowed to look more authoritative than it is.

6. Deployment topology (local)

Everything runs via `infra/docker-compose.yml`: one Redis container, one container per backend service, one frontend container. Host port mappings exist only to dodge two local conflicts on this dev machine (see `CLAUDE.md`'s "Known environment quirks") — internal container-to-container calls always use the plain service name and port 8000 (`http://knowledge-botanical:8000`, etc.), regardless of what's mapped on the host. There is no reverse proxy, service mesh, or ingress controller in front of these services locally — the gateway *is* the ingress point, doing CORS + rate limiting + proxying in application code rather than infrastructure. (See `docs/TECHNICAL_GUIDE.md` for the trade-off discussion of when that stops being enough and something like Istio becomes worth the operational cost.)

Phase 2 (deferred by design, tracked in `CLAUDE.md`'s open items and `application_design_v2...md` §9): actual cloud deployment manifests, live fallback to IMPPAT/ChEBI/PubChem/CTD on a Tier 1 cache miss, multi-symptom conflict resolution, personalized history across sessions.

7. Language migration status (Python → Java)

Scope, decided explicitly and re-derived from the code (not assumed):

- **Stays Python:** `agent-intake`, `agent-mapping`, `agent-explanation` — the only three services that `import shared.llm` / call Anthropic.
- **Moves to Java (Spring Boot):** everything else — `knowledge-botanical`, `knowledge-compound`, `knowledge-toxicology`, `knowledge-rules`, `agent-retrieval`, `agent-safety`, `agent-scoring`, `agent-reporting`, `email`, `orchestrator`, `gateway`.
- **Frontend:** stays Next.js/React — not part of this migration.
- **Approach:** one service at a time, each verified end-to-end against the rest of the (still-mostly-Python) running stack before moving to the next, rather than a big-bang rewrite.

Service	Status
<code>knowledge-botanical</code>	In progress — first slice, Spring Boot 3
<code>knowledge-compound</code> , <code>-toxicology</code> , <code>-rules</code>	Not started (same shape as botanical)
<code>agent-retrieval</code>	Not started
<code>agent-safety</code> , <code>agent-scoring</code> , <code>agent-reporting</code>	Not started
<code>email</code>	Not started
<code>orchestrator</code> , <code>gateway</code>	Not started (last — most central, highest blast radius)

See [docs/TECHNICAL_GUIDE.md](#) for why each Java-side dependency choice was made, and [docs/DEVELOPER_GUIDE.md](#) for the concrete steps to migrate the next service following the same pattern.